Feature-to-Code Subsystem

Conceptual Design Document

Subsystem of an Autonomous Software Engineering System

Draft — March 2026

1. Purpose and Scope

This document describes the conceptual design of the **Feature-to-Code subsystem**: the component of an autonomous software engineering system that takes a scoped work unit and produces source code and unit tests committed to a repository.

The subsystem does not receive a full feature architecture directly. An upstream decomposition agent consumes the feature architecture document and breaks it into a sequence of scoped slices, each representing an independently deployable unit of work (typically corresponding to a PR or a set of commits). The decomposition agent has full access to the codebase and determines the sequencing, dependency ordering, and scope boundaries of each slice.

This subsystem receives a single slice as its input. Each slice is treated as a trusted input that defines a scoped behavioral intent, the relevant subset of requirements, scope boundaries, assumptions, and dependencies on prior slices. The subsystem processes one slice per invocation. When the decomposition agent identifies slices that have no mutual dependencies, multiple instances of this subsystem may run in parallel on independent slices.

The subsystem's outputs—source code, database schemas, API specifications, configuration files, and unit tests—are consumed by a downstream integration, QA, or deployment subsystem. This design therefore defines a clean output contract but does not address downstream acceptance.

Human involvement is limited to fallback escalation. The system operates autonomously unless it detects ambiguity, conflict, or conditions that exceed its confidence thresholds. It must support both greenfield features and integration into existing codebases with established patterns.

> **Design Constraint from Strategy**
> This subsystem is designed as an application of the broader strategy for autonomous software engineering. Every structural choice—artifact flow, role separation, rubrics, escalation gates—traces to a specific guiding policy from that strategy. Traceability to policy is noted throughout.

2. Artifact Flow

The subsystem is structured around a linear sequence of artifacts with defined handoffs. Each stage consumes concrete artifacts and produces concrete artifacts. Progress is measured by the presence and quality of these artifacts, not by self-reported status. This carries out **Policy 1** (control problem), **Policy 2** (externally checkable outcomes), and **Policy 6** (learning).

Stage	Artifact Produced	Produced By	Accepted By
1. Intake	Validated slice + codebase context snapshot	Intake Analyst	Spec Author (implicit)
2. Specification	Implementation specification (behavioral contract, scope, constraints)	Spec Author	Spec Reviewer
3. Test Design	Unit test suite (code, not prose)	Test Designer	Test Reviewer
4. Implementation	Source code, schemas, config, API specs	Implementer	Code Reviewer
5. Evaluation	Evaluation report (test results, coverage, static analysis, rubric scores)	Evaluator	Gate Controller
6. Decision	Accept, revise, escalate, or halt with error signal + learning record	Gate Controller	—

Each artifact is a concrete, inspectable file or structured record in the repository. The flow is enforced by workflow tooling: stage N+1 cannot begin until stage N's artifact has been produced and accepted by the designated reviewer.

2.1 Stage Descriptions

Stage 1: Intake

The Intake Analyst receives a single scoped slice from the upstream decomposition agent. The slice defines the behavioral intent, scope boundaries, assumptions, and dependencies for one independently deployable unit of work. The Intake Analyst gathers a codebase context snapshot relevant to this slice: directory structure, existing schemas, API contracts, test patterns, and coding conventions that the slice interacts with. For greenfield features, this snapshot is either empty or derived from a project template.

The output is a validated intake package. Validation means checking the slice for internal completeness (behavioral intent is defined, scope boundaries are clear, dependencies on prior slices are stated) and checking it against the codebase context for consistency (stated

assumptions about existing state actually hold). If the Intake Analyst detects that a slice's assumptions are invalid—for example, a dependency on a schema state that does not yet exist in the codebase—the subsystem halts processing of this slice and emits a structured error signal to the decomposition agent (see Section 6.3). If a different kind of critical ambiguity is detected that cannot be resolved from the slice or codebase, the system escalates to a human.

Stage 2: Specification

The Spec Author consumes the intake package and produces an implementation specification. This is the behavioral contract for the feature: it defines what the code must do, what it must not do, boundary conditions, error handling expectations, and integration constraints with existing systems.

The specification is shaped by change type, following the strategy's guidance. For new features, it emphasizes behavior and edge cases. For changes to existing code, it additionally defines behavior-preservation boundaries. The specification constrains outcomes without over-prescribing internal implementation.

The Spec Reviewer checks the specification against the slice for fidelity, against the codebase context for feasibility, and against the specification rubric for quality. The Spec Reviewer is a distinct role from the Spec Author.

Stage 3: Test Design

The Test Designer consumes the accepted specification and produces a unit test suite—executable test code, not prose descriptions. Tests are written against the specification's behavioral contract. Each test traces to a specific clause or requirement in the specification.

The Test Designer does not see or produce any implementation code. This is a structural separation: the Test Designer's repository permissions grant read access to the specification and write access to the test directory only. This prevents test design from being shaped by implementation convenience.

The Test Reviewer checks tests against the test rubric and specification traceability. The Test Reviewer is distinct from the Test Designer.

Stage 4: Implementation

The Implementer consumes the accepted specification and the accepted test suite, then produces source code, database schemas, API specifications, configuration files, and any other code-level artifacts defined in the specification's scope.

The Implementer has read access to the specification and tests but cannot modify either. Write access is limited to implementation directories. This means the Implementer's path of least resistance is to write code that satisfies the existing tests and specification, not to weaken the constraints.

The Code Reviewer checks the implementation against the code rubric and the specification. The Code Reviewer is distinct from the Implementer.

Stage 5: Evaluation

The Evaluator runs the full evaluation suite and produces a structured evaluation report. This report includes test pass/fail results, code coverage metrics, static analysis findings, and rubric scores for all quality-critical artifacts produced during this cycle.

The Evaluator does not produce or modify any of the artifacts being evaluated. The Evaluator's role is to generate evidence, not to interpret whether it is sufficient. Interpretation is the Gate Controller's responsibility.

Stage 6: Decision

The Gate Controller consumes the evaluation report and makes one of four decisions: accept (artifacts are committed to the output branch), revise (specific artifacts are sent back to their producing role with concrete feedback), escalate (the system requests human intervention, with a structured summary of what is unresolved and why), or halt with error signal (the subsystem stops work on this slice and reports to the decomposition agent that a slice assumption has failed—see Section 6.3).

Every decision is recorded as a learning record: what was decided, why, which evaluation evidence was decisive, and what the system would need to see in a revision. These records accumulate to support the learning objective of **Policy 6**.

3. Roles, Authority, and Permissions

Each role has a narrow set of aligned objectives, limited permissions, and explicit boundaries. This carries out **Policy 1** (control structure) and **Policy 3** (separation of powers). No role both produces and accepts the same class of artifact.

Role	Writes	Reads	Cannot Access	Reviews	Accepts
Intake Analyst	Intake package	Slice, codebase	—	—	—
Spec Author	Impl. specification	Intake package, codebase	Tests, impl. code	—	—
Spec Reviewer	—	Slice, intake, spec	Tests, impl. code	Specification	Specification
Test Designer	Test suite	Specification	Impl. code	—	—
Test Reviewer	—	Specification, tests	Impl. code	Test suite	Test suite

Implementer	Source code, schemas, config	Specification, tests, codebase	—	—	—
Code Reviewer	—	Specification, tests, code, codebase	—	Code	Code
Evaluator	Evaluation report	All artifacts (read-only)	—	—	—
Gate Controller	Decision + learning record	Evaluation report, all artifacts	—	—	Final output

3.1 Permission Enforcement

Permissions are not advisory. They are enforced structurally through repository branch protections, directory-level write permissions, and workflow gates. An Implementer that attempts to modify test files is blocked by the system, not merely instructed not to. This follows the strategy's design principle: make the desired outcome the easiest equilibrium.

Key Separation: Test Designer ↔ Implementer

The Test Designer cannot see implementation code. The Implementer cannot modify tests. This is the subsystem's most critical separation of powers. It ensures that tests are derived from the specification's behavioral contract rather than reverse-engineered from implementation details, and that the Implementer's cheapest path to success is writing correct code rather than weakening test assertions.

3.2 Role-to-Agent Mapping

Each role is a logical function, not necessarily a distinct LLM instance. Multiple roles may be served by the same underlying model, provided that permission boundaries are enforced externally (the model cannot access artifacts outside its role's read/write scope for that stage). The separation is structural, not reliant on the model's self-governance.

This is an important implication of the strategy's diagnosis that LLM self-governance degrades where quality gradients are shallow. Telling a model "do not look at the implementation" is less reliable than giving it a session in which implementation files are not present. The structural separation compensates for a domain in which "follow these quality standards" is an underdetermined instruction, not for models that cannot follow instructions at all.

4. Rubrics for Quality-Critical Artifacts

Rubrics convert implicit quality expectations into checkable criteria. They are used by reviewers and the Evaluator to assess artifacts against a consistent standard. This carries out **Policy 2** (externally checkable outcomes) and **Policy 6** (learning).

4.1 Implementation Specification Rubric

Criterion	Standard
Completeness	Every behavioral requirement from the feature architecture is addressed. Scope boundaries are explicit.
Precision	Acceptance criteria are specific enough to write a pass/fail test against. No vague terms (“should handle errors gracefully”) without concrete definition.
Constraint Clarity	Integration constraints (existing schemas, API contracts, coding conventions) are enumerated when an existing codebase is involved.
Boundary Definition	Edge cases, error conditions, and out-of-scope behaviors are explicitly stated.
Traceability	Each clause traces to a requirement in the slice.
Non-Prescriptiveness	The specification defines outcomes, not internal implementation, unless a design constraint is intentional and justified.

4.2 Unit Test Suite Rubric

Criterion	Standard
Specification Traceability	Every test traces to a specific clause in the implementation specification. No untethered tests.
Assertion Precision	Assertions check specific expected values or behaviors, not broad truthiness. Each assertion has a clear failure message.
Edge-Case Coverage	Boundary conditions, error paths, and off-nominal inputs defined in the specification have corresponding tests.
Narrow Error Expectations	Tests that expect errors specify the exact error type/message, not broad exception classes.
Independence	Tests do not depend on execution order or shared mutable state.

Readability	Test names describe the behavior being verified. Test structure follows arrange-act-assert or equivalent.
Non-Implementation-Coupling	Tests assert on observable behavior (inputs, outputs, side effects), not internal state or implementation details.

4.3 Source Code Rubric

Criterion	Standard
Specification Compliance	All behavioral requirements in the specification are implemented. No unspecified behaviors are introduced.
Test Passage	All accepted unit tests pass without modification to the tests.
Codebase Consistency	Code follows existing conventions (naming, structure, patterns) when integrating into an existing codebase. For greenfield, follows the template or stated conventions.
Schema/API Contract Fidelity	Database schemas and API specifications match the specification's constraints exactly.
Static Analysis	No new warnings or errors from configured static analysis tools.
Minimal Scope	No code changes outside the specification's defined scope. No opportunistic refactors unless explicitly in scope.

5. Independent Pressure and Risk Focus

The strategy directs the strongest independent checking pressure at the defect class that is most dangerous at the current stage. For this subsystem, the default highest-risk defect class is **behavioral incorrectness**: code that does not do what the feature architecture intended. This carries out **Policy 4**.

5.1 Where Independent Pressure Is Applied

The subsystem applies structured independence at three points, listed in order of priority:

- **Test-Implementation separation.** The Test Designer and Implementer cannot see each other's work during production. Tests are written from the specification, not from the code. This is the strongest independence boundary in the subsystem and directly targets behavioral correctness.

- **Review independence.** Every quality-critical artifact (specification, tests, code) is reviewed by a role distinct from its author. Reviewers check against rubrics and upstream artifacts, not author self-reports.
- **Evaluation independence.** The Evaluator produces evidence (test results, coverage, static analysis). The Gate Controller interprets that evidence. Neither role produced the artifacts being evaluated.

5.2 Adapting Pressure Over Time

The default emphasis on behavioral correctness may shift as the product matures. If learning records show that behavioral defects are rare but structural degradation or performance regressions are frequent, the Gate Controller's acceptance criteria and the Evaluator's suite should be reconfigured to apply stronger pressure on those defect classes. The architecture supports this without structural redesign: the rubrics, evaluation tools, and acceptance thresholds are parameters, not hardcoded.

6. Human Escalation Model

Humans are a fallback, not a checkpoint. Escalation occurs only when the system detects conditions it cannot resolve autonomously. The system is designed to escalate early and with structure, not to push forward under false certainty.

6.1 Escalation Triggers

- **Ambiguity in the slice** that the Intake Analyst cannot resolve from the slice itself or the codebase context. Example: contradictory requirements, undefined behavior for a critical edge case. (Note: if the ambiguity is a sequencing or decomposition problem rather than a content problem, the subsystem emits an error signal to the decomposition agent per Section 6.3 instead of escalating to a human.)
- **Specification conflict with codebase constraints** that the Spec Author or Spec Reviewer identifies as unresolvable within the current scope. Example: the feature requires a schema change that would break an existing contract.
- **Repeated revision failure.** If a stage artifact fails review or evaluation more than a configurable threshold of times (default: two), the Gate Controller escalates rather than continuing to loop.
- **Confidence below threshold.** Any role that detects a condition where its confidence in the artifact it is producing falls below a defined threshold should flag this. The Gate Controller aggregates these flags.

6.2 Escalation Format

Every escalation produces a structured record containing: the specific question or conflict, the artifacts and evidence that led to the escalation, what the system has already tried, and what resolution would allow the system to continue. This minimizes the human's cognitive load and avoids vague requests for help.

6.3 Slice Assumption Failure: Error Signal to Decomposition Agent

This is distinct from human escalation. When the subsystem discovers during any stage that a slice's stated assumptions do not hold—a dependency on prior state that does not exist, a scope boundary that turns out not to be independently deployable, a constraint conflict that is a property of the sequencing rather than the slice's content—the subsystem halts work on that slice and emits a structured error signal to the upstream decomposition agent.

The error signal contains: which assumption failed, the evidence that revealed the failure, the stage at which it was detected, and any partial work completed before the halt. The subsystem does not attempt to fix the sequencing problem or adjust its own in-progress work. It stops cleanly and reports.

The decomposition agent receives this signal and is responsible for re-planning the remaining slices. It may revise the failed slice's scope, reorder remaining slices, or merge slices. Once re-planning is complete, the decomposition agent may resubmit a corrected slice to a new invocation of this subsystem.

This is deliberately not a full feedback loop. The subsystem does not maintain state between invocations or attempt to resume halted work. Information crosses the boundary (the error signal), but control authority does not: the subsystem does not influence re-planning, and the decomposition agent does not intervene in the subsystem's internal flow. This keeps the boundary between decomposition and execution clean.

7. Traceability

The subsystem maintains explicit links between artifacts at every stage. This carries out **Policy 2** (auditable success claims) and **Policy 6** (cumulative learning).

- Slice requirements → specification clauses.
- Specification clauses → individual tests.
- Tests → code locations they exercise.
- Evaluation findings → specific artifacts and rubric criteria.
- Gate decisions → evaluation evidence that drove them.
- Revisions → the feedback that prompted them.

Traceability is stored as structured metadata alongside artifacts in the repository, not as free-text commentary. This makes it queryable by downstream subsystems and by the learning loop.

8. Learning Loop

Each cycle through the subsystem produces not just code and tests but a learning record. This carries out **Policy 6** (optimize for learning over throughput).

8.1 What Gets Recorded

- Ambiguities discovered in the slice and how they were resolved (or escalated, or signaled to the decomposition agent).
- Specification clauses that caused implementation difficulty or test failure, and why.
- Defect patterns: which rubric criteria failed most often, at which stages.
- Revision counts per stage and per artifact class.
- Escalation causes and outcomes, and slice assumption failure signals sent to the decomposition agent.
- Codebase integration friction: conventions that were hard to detect or follow.

8.2 How Learning Feeds Back

Learning records are consumed in three ways. First, they inform the Intake Analyst on subsequent slices: known ambiguity patterns, common integration friction points, and codebase conventions are surfaced as context. Second, they inform the upstream decomposition agent: slice assumption failures and recurring integration friction help improve decomposition quality for future features. Third, they inform architectural adaptation of this subsystem: if a specific defect class persists across multiple cycles, the system should consider adding or adjusting rubric criteria, evaluation tools, or role emphasis in response.

Architectural changes driven by learning records are treated as empirical hypotheses per **Policy 5**: state the problem, define the assumption, monitor whether it helps, and remove or revise if it does not.

9. Greenfield vs. Existing Codebase Support

The subsystem handles both cases through the codebase context snapshot gathered at Intake. The differences are confined to a few specific points in the flow:

Aspect	Greenfield	Existing Codebase
Intake	Codebase context is empty or template-derived.	Codebase context includes directory structure, schemas, API contracts, conventions, and relevant existing tests.
Specification	Fewer integration constraints. Focus on behavioral contract.	Must enumerate integration constraints and behavior-preservation boundaries.
Test Design	No existing test patterns to match.	Must follow existing test conventions. May need to account for existing fixtures or utilities.
Implementation	Full freedom on internal structure within spec constraints.	Must follow existing naming, patterns, and architectural conventions.
Code Rubric	Consistency criterion uses stated conventions or template.	Consistency criterion checks against actual codebase patterns.

10. Output Contract

The subsystem's output, delivered to the downstream integration/QA/deployment subsystem, consists of:

- **Source code** (including schemas, API specifications, configuration files) committed to a defined branch in the repository.
- **Unit test suite** committed alongside the source code, all passing.
- **Implementation specification** committed as a reference artifact for the downstream subsystem.
- **Evaluation report** documenting test results, coverage, static analysis, and rubric scores.
- **Traceability metadata** linking slice → specification → tests → code.
- **Learning record** for the cycle.

Alternatively, if the subsystem halts due to a slice assumption failure, the output is:

- **Error signal** containing the failed assumption, the evidence, the stage at which the failure was detected, and any partial work completed.

- **Learning record** for the halted cycle.

On the success path, the downstream subsystem can rely on the following guarantees: all unit tests pass, all rubric criteria have been evaluated and scored, the specification and slice are traceable through the output, and any unresolved ambiguities have been escalated and resolved (or the slice was not accepted). On the error path, the decomposition agent can rely on a structured diagnosis of why the slice could not be processed, sufficient to inform re-planning without re-running the subsystem's analysis.

11. Anti-Patterns This Design Avoids

The following failure modes, identified in the strategy, are structurally mitigated by this design:

Anti-Pattern	Structural Mitigation
One role defines intent, designs tests, implements, and judges success	Six distinct stages with separate producing and accepting roles. No role both writes and accepts the same artifact.
Weak or implicit quality criteria	Explicit rubrics for specifications, tests, and code. Rubric criteria are checkable, not subjective.
Agents modifying the artifacts that constrain them	Permission enforcement: Implementer cannot edit tests or spec. Test Designer cannot see implementation.
Excessive agent proliferation	Nine roles is the minimum that achieves the required separations. Roles may share underlying models. Complexity is earned by the control gaps they close.
Architectures optimized for passing signals rather than correctness	Evaluation is based on externally checkable outcomes (test results, coverage, static analysis), not self-reported completion.
Relying on cooperation where independence is required	Test–Implementation separation and review independence are enforced by permissions, not by instructions to cooperate.

12. Design Rationale Summary

This subsystem design follows the strategy's core framing: autonomous software engineering is a control problem under uncertainty, not just a generation problem. The artifact flow, role

separation, rubrics, and evaluation structure are all oriented toward the objective of maximizing trustworthy software progress per unit of time and compute.

The design applies the strategy's equilibrium test: when an agent in this system faces pressure, its easiest path to success is producing a better artifact that survives independent checks, not gaming a metric. Permissions make constraint-weakening harder than compliance. Rubrics make quality visible rather than implicit. Separated evaluation makes self-serving interpretation structurally impossible.

The design also follows the strategy's empirical stance: this architecture is a starting hypothesis. As learning records accumulate, the system should adapt its rubrics, evaluation emphasis, and role structure in response to observed defect patterns, not in response to theoretical preferences.